MagicCode Super-Resolution Software Specification

Document Version: v3.0.0. Release Date: October 24, 2026.

1. Document Overview

This specification defines capabilities, platforms, and functional boundaries of MagicCode Super-Resolution Software v3.0.0. It covers image and graphic GPU paths, image CPU paths, and Vulkan frame generation (GEN).

1.4 Document Status

Version	Date	Status	Change
v2.0.0	September 2, 2026	Superseded	Graphic modes, session-API contract, platform matrix.
v2.1.0	September 4, 2026	Superseded	MC_Enable/MC_Disable public API (removed in v2.2.0).
v2.2.0	September 30, 2026	Superseded	nscaler public API (mc_nscaler_enable/control/disable/version); Linux x86_64 CPU image archive; Linux Vulkan Graphic (Vulkan Temporal) validated from source; image (1,2) band routing update.
v3.0.0	October 24, 2026	Current	Two-frame mc_nscaler_enable; alg_mode rename + GEN bits; Vulkan FG; Linux archive includes Vulkan Graphic (Vulkan Temporal)/TAAU + FG; version-string lifetime.

2. System Requirements

Platform	GPU backends	Graphic
Android	Vulkan, OpenGLES	Graphic GPU yes; CPU graphic no; FG not in this Android Vulkan build
iOS / iPadOS / macOS	Metal	Graphic GPU yes; CPU graphic no; FG not available (Metal)
Windows	Vulkan, Direct3D 11, OpenGL 4.3	Graphic GPU yes; CPU graphic no; FG on Vulkan
Linux x86_64	Vulkan	CPU image + Vulkan Graphic (Vulkan Temporal)/TAAU + FG in product .a/.so; CPU graphic no

- Input size [64, 4032]. Image scale [1, 8]. Graphic scale (1, 8].

- CPU image: x86_64 AVX2 and ARM64 NEON buffer paths. CPU graphic is not available.
- Linux x86_64: lib/linux/libmagic_sr.a and libmagic_sr.so include CPU image (AVX2/FMA) plus Vulkan Graphic (Vulkan Temporal)/TAAU and FG (MAGIC_SR_HAS_FG). Rebuild with tools/build_linux_lib.sh; link -lm -lpthread. CPU graphic is unavailable.

3. Functional Capabilities

Value	Enumerator	Role
0	SR_SPEED_MODE_FOR_IMAGE	Image, throughput first
1	SR_BALANCED_MODE_FOR_IMAGE	Image, quality/cost trade-off
2	SR_SPEED_MODE_FOR_GRAPHIC	Graphic, throughput first
3	SR_BALANCED_MODE_FOR_GRAPHIC	Graphic, canonical FSR-family path
4	GEN_SPEED_MODE_FOR_GRAPHIC	FG-Fast (analytic, no weights). Vulkan + MAGIC_SR_HAS_FG. Combinable with SR bits; bare 4 equals SR_SPEED_MODE_FOR_IMAGE|GEN_SPEED.
8	GEN_BALANCED_MODE_FOR_GRAPHIC	FG-CoreV3. Same availability as Fast. Needs magic_fg_core.bin. Do not set both GEN bits.

Image modes upscale a single frame. Graphic modes accumulate history and require depth, motion, jitter, and camera parameters through mc_nscaler_input_frame_t / temporal_frame_t. GEN modes on Vulkan interpolate t=0.5 between two frames when in_frame[1] is non-NULL.

- FG (GEN_SPEED / GEN_BALANCED): Vulkan + MAGIC_SR_HAS_FG only. Windows and Linux product libraries include it. Android Vulkan currently does not. Other backends return -100050 / -102013.
- GEN_BALANCED uses ctrl_param_t.model_path as the FG weight file (typically model/magic_fg_core.bin, 331,136 bytes). Empty path uses MAGIC_FG_WEIGHT_PATH, working-directory magic_fg_core.bin, then model/magic_fg_core.bin. Recreate the handle to change weights.
- Vulkan Speed (Lite-TAAU) now lets an explicit strong reactive mask suppress the still-camera accumulation branch. Previously that branch ignored the caller reactive mask when the camera was still or per-pixel MV was below about 0.2 px, so pixels with ~0 MV but changing content (TV / video-wall / LED-screen textures) could keep ghosting for about 64–96 frames after a content cut. Strong reactive (≥ about 0.85, fully effective at ≥ 0.95) now weakens that accumulation. Omitting reactive or passing a weak value (< 0.85) is unchanged. Recommendation: mark screen-like objects whose MV is 0 but whose content changes (for example with CustomDepth stencil) as reactive at strength 1.0. Limit: Vulkan only in this release. The Metal / GLES / D3D11 Speed still-accumulation structure is the same idea but is not modified or verified here.

Graphic Super-Resolution

Graphic super-resolution runs through the nscaler session API: libmagic_sr.a (Windows: libmagic_sr.lib) + mc_interface.h + mc_nscaler_enable / mc_nscaler_control / mc_nscaler_disable. v2.2.0 replaced the v2.0.0 MC_Init / MC_Process / MC_Uninit entry and removed the v2.1.0 MC_Enable helper (mc_enable.h is deleted). A NULL *handle on mc_nscaler_enable creates a session with defaults (2x SR_SPEED_MODE_FOR_IMAGE) and processes the first frame. Pass mc_nscaler_input_frame_t *in2[2] = { &in, NULL } (set in2[1] only for GEN interpolate).

Modes

Value	Enumerator	Role
0	SR_SPEED_MODE_FOR_IMAGE	Image, throughput first
1	SR_BALANCED_MODE_FOR_IMAGE	Image, quality/cost trade-off
2	SR_SPEED_MODE_FOR_GRAPHIC	Graphic, throughput first
3	SR_BALANCED_MODE_FOR_GRAPHIC	Graphic, canonical FSR-family path
4	GEN_SPEED_MODE_FOR_GRAPHIC	FG-Fast (analytic, no weights). Vulkan + MAGIC_SR_HAS_FG. Combinable with SR bits; bare 4 equals SR_SPEED_MODE_FOR_IMAGE|GEN_SPEED.
8	GEN_BALANCED_MODE_FOR_GRAPHIC	FG-CoreV3. Same availability as Fast. Needs magic_fg_core.bin. Do not set both GEN bits.

- Graphic scaler_factor is (1, 8]; 1.0 itself is rejected. Image remains [1.0, 8.0].
- SR_SPEED_MODE_FOR_GRAPHIC on Metal, Vulkan, and OpenGLES currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family graphic path (same family as Balanced, not a separate customer API).
- SR_BALANCED_MODE_FOR_GRAPHIC uses the canonical FSR-family path on every GPU graphic backend. Desktop Balanced edge-resolve is supported on Windows Vulkan, Windows Direct3D 11, desktop OpenGL 4.3, and Apple Metal. Android Vulkan and Android OpenGLES do not run edge-resolve. If enable_sharpening is on, RCAS runs after edge-resolve.
- CPU backends (X86 / NEON) cannot run graphic modes.
- The API accepts (1, 8]. Exact 2× has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

Minimum per-frame inputs

- mc_nscaler_input_frame_t / mc_nscaler_output_frame_t: caller-owned color and output GPU resources.
- mc_nscaler_input_frame_t.frame: non-NULL temporal_frame_t* with struct_size = sizeof(temporal_frame_t).
- temporal_frame_t.depth and motion at input resolution.
- jitter_offset_x/y in input pixels, top-left, +X right, +Y down.
- frame_index / reset_history; camera_near / camera_far / camera_fov_y (radians).

- gpu_context at create (mc_nscaler_control SET_PARAM, or the first mc_nscaler_enable): Vulkan requires physical_device + device; D3D11 requires device; Metal device may be NULL; GL/GLES use the current context (library never makes current).
- Vulkan graphic requires mc_nscaler_input_frame_t.command_buffer to be a recording VkCommandBuffer. Metal command_buffer is optional. D3D11 / GL / GLES leave it NULL.
- Create-stage depth_reversed / depth_infinite / hdr_color on ctrl_param_t (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.
- motion_vector_scale_x/y is the only MV numeric conversion. (0,0) defaults to (input_width, input_height). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.
- reactive / transparency are optional. Explicit caller textures win per channel.
- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing reactive and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing reactive only and bind an internal zero transparency texture.
- Both graphic modes enable depth+MV history rejection by default.
- Diagnostic only: MAGICSR_TAAU_AUTO_MASK=0 and MAGICSR_TAAU_HIST_REJECT=0 can turn those defaults off. They are not a customer configuration path.

4. Resource and synchronization contract

- Caller owns application GPU objects. The library owns internal history, scratch, and pipelines only.
- Vulkan: library does not begin/end/submit/wait. Restore of input layouts after compute; output remains GENERAL.
- Complete prior GPU work before mc_nscaler_disable or a size-changing enable.

5. Engine plugins

Unity and Unreal plugins expose Enable/Disable for image GPU textures. Full graphic integration is the Native session API described above. Plugin Enable is not a complete graphic entry.

6. Conformance notes

API support is not the same as device-run verification. Exact 2× has optimized graphic kernels; other rates in (1, 8] are supported by the API. Do not claim every scale or GPU vendor is verified.